Documentation of Avinash Group

For a developer new to `avinashgroup_app`. Read **Start Here** first (the mental model and where things live), then use the **Reports** catalog and the **Customizations & Portal** section as a reference. Everything below reflects the current code.

How to read this: - **Reports** are short — What / Source / Filters / PDF, plus a ⚠️ gotcha when there is one. - **Customizations & Portal** are fuller — Purpose / How it works / Files / gotcha — because these are the parts you actually have to *understand and change*. - The shared conventions (company scoping, BS dates, number format, PDF patterns) live once in **Start Here**, so the per-item entries stay short.

Start Here

Where things live

Area	Path	What's there
Reports	`avinash_group_app/report/<name>/`	Each has `.py` (query/rows), `.js` (filters + print), `.json` (config), and often `<name>_pdf.html` (PDF template).
Portal pages	`templates/pages/<name>.py` + `.html`	Customer/supplier-facing web pages (e.g. Customer Statement).
Shared business logic	`custom_code/`	Server-side handlers grouped by area: `common/` (taxes), `SalesInvoice/`, `purchase_invoice/`, `Override/` (ERPN `globalfilter/`, `workflow*`.
Client scripts	`public/js/`	Form behaviour (VAT/TDS calc, warehouse, numbering, print helpers).
Custom fields / doctypes	`avinash_group_app/doctype/` and `avinash_group_app/custom/*.json`	New doctypes + custom fields on core doctypes.
Wiring (the glue)	`hooks.py`	`app_include_js`, `doctype_js`, `doc_events`, `override_whitelisted_methods`, `override_doctype_class` **Start here to see how a feature is connected.**

Conventions every report/feature shares

- **Reports are Script Reports** (`execute(filters)` builds rows in Python). Most build their own bold total rows (`add_total_row = 0`). All run **synchronously** (`prepared_report = 0`).

  > ⚠️ Frappe auto-switches a report to background (prepared) mode if one `execute()` takes
  >
  > > 15s. Heavy queries are written to stay under that (e.g. hit indexed columns first).

- **Company scoping:** filter dropdowns come from whitelisted helpers — `get_company_customers`, `get_company_suppliers`, `get_company_items`, `get_company_customer_groups`, `get_company_party_groups`, `get_company_bank_accounts` — scoped via the master's `custom_company` field.
- **Nepali (BS) dates:** the "Miti" column comes from custom fields (`custom_invoice_miti`, `custom_nepali_miti`, `custom_posting_miti`), usually `SUBSTRING_INDEX(..., ' ', 1)`.
- **Numbers:** `_fmt_inr` = Indian lakh/crore grouping (`en_IN`, fallback `{:,.2f}`); `_fmt_qty` = 3 dp; currency labelled NPR.
- **Console / setup commands:** some features are applied **manually per site** (not by migrate). Where relevant, the feature's entry has a **Setup / Backfill (console)** block. Run it with `bench --site <site> console` (paste the import + call) or, for an arg-less function, `bench --site <site> execute <dotted.path.to.function>`. Each function commits itself; re-run on every site (dev/live), and `bench restart` on live if it changes forms.

PDF / Print — three patterns

Pattern	Used by	How it works
A — custom template	Party Ledger, Party Ledger Summary, Sales/Purchase Register, Receipt Register, Sales Analysis ×3	Whitelisted `download_pdf` re-runs `execute`, paginates in Python, renders `*_pdf.html`, `get_pdf`. In-body `Page X of Y` (works on plain wkhtmltopdf). Print is overridden to call `download_pdf` (`view=1`, inline) → Print and Download give the same PDF. "Pick Columns" supported.
B — shared helper	Advance Tax TDS, Loan Summary	No local template. `public/js/report_print_orientation.js` (loaded globally) adds the orientation dialog + portrait CSS + PDF styling.
C — stock Frappe	One Lakh Above	Built-in query-report Print / PDF / Export.

Reports — Quick Reference

#	Report	Source	Purpose
1	Party Ledger	GL Entry	Tally-style party statement: opening → txns → closing.
2	Party Ledger Summary	GL Entry	One line/party: opening / debit / credit / closing.
3	Sales Register	Sales Invoice	Per-invoice VAT register (tax-free / export / taxable / VAT).
4	Purchase Register	Purchase Invoice	Per-invoice VAT register (tax-free / taxable / import / capitalized).
5	Receipt Register	Payment Entry	Customer receipts in 3 views.
6	SA — Customer Summary	Sales Invoice	Per-customer qty/value, optional returns/net.
7	SA — Customer Details	Sales Invoice	Per customer → product (UOM).
8	SA — Product Details	Sales Invoice	Per product → invoice → customer.
9	Advance Tax TDS	Purchase Invoice	Supplier TDS withholding by category.
10	Loan Summary	GL Entry	Company-wise loan closing balances.
11	One Lakh Above	SI + PI	Parties with totals ≥ NPR 100,000.

`*` = required filter. PDF patterns A/B/C are defined in **Start Here**.

Reports

1. Party Ledger

- **What:** Ledger for one party, or grouped multi-party when 0 / 2+ selected. Powers `/customer_statement`.
- **Source:** `tabGL Entry` (`is_cancelled=0`, company, party_type); `LEFT JOIN` SI/PI for description. Running balance computed in Python; rows merged per voucher.
- **Filters:** `company*`, `party_type`, `party` (1→flat, 0/2+→grouped), `account`, `from*` / `to*`, `voucher_no` (LIKE), `show_remarks`, `show_balance`, `detailed_mapping`, `show_contract_form` (**default ON**).
- **PDF:** A — `party_ledger_pdf.html`, default Landscape.
- **Account exclusion (gated):** accepts optional `exclude_account_patterns` (`account_name` LIKE list) → `gle.account NOT IN (matching accounts)`, applied to opening + period rows. The desk report never sets it; the Customer Statement portal uses it to hide deposit/security accounts.

> ⚠️ One row per voucher × party → large for wide ranges. Enrichment (Miti, remarks, names) is batched in `IN` chunks of 500 to avoid N+1.

2. Party Ledger Summary

- **What:** One aggregate line/party. Modes: **Super Summary** (flat + grand total) / **Group Wise** (per-group subtotals).
- **Source:** `tabGL Entry GROUP BY party` ; `opening = SUM(debit-credit) WHERE posting_date < from_date OR is_opening='Yes'` .
- **Filters:** `report_type*` , `company*` , `party_type*` , `party_group` , `party` , `from*` / `to*` , `show_zero_balance` , `closing_drcr` (DB/CR).
- **PDF:** A — default Portrait. Excel export overridden (Opening/Closing as magnitude + Dr/Cr column).

3. Sales Register

- **What:** One row / submitted Sales Invoice; VAT split tax-free / export / taxable / VAT. Returns view (`abs()`).
- **Source:** `SI ⋈ SII ⋈ Customer` , `GROUP BY si.name` . Buckets = conditional `SUM` over `sii.custom_vat_apply_on` + `c.territory` .
- **Filters:** `company` , `from*` / `to*` , `customer` , `is_return` .
- **PDF:** A — Landscape, Pick-Columns.

> ⚠️ `company` / `customer` are string-interpolated into the SQL; dates are bound params.

4. Purchase Register

- **What:** One row / Purchase Invoice; buckets tax-free / taxable / import / capitalized + VAT + qty.
- **Source:** `PI ⋈ PII ⋈ Supplier ⋈ Item` , `GROUP BY pi.name` . Buckets via `custom_vat_apply_on` + `is_fixed_asset` + `custom_territory` .
- **Filters:** `company` , `from*` / `to*` , `supplier` , `purchase_type` , `is_return` .
- **PDF:** A — Landscape; shares the "Pick Columns" patch with Sales Register.

5. Receipt Register

- **What:** Customer receipts (Payment Entry, `payment_type='Receive'`). 3 views: Date Wise / Customer Wise / Summary.
- **Source:** `tabPayment Entry` only; `docstatus=1` , `party_type='Customer'` .
- **Filters:** `view*` , `company*` , `from*` / `to*` , `customer` , `bank` .
- **PDF:** A — orientation auto per view.

> ⚠️ "GL Code" column is an unfinalized placeholder; cheque no `"1"` is suppressed.

6. Sales Analysis — Customer Summary

- **What:** Per-customer qty + gross value (incl. excise); optional returns/net.
- **Source:** `SI ⋈ SII ⋈ Customer` , `GROUP BY customer` ; `value = SUM(amount + custom_excise_value)` ; returns negated to positive.
- **Filters:** `company` , `from*` / `to*` , `customer` , `customer_group` , `item_code` , `include_return` .
- **PDF:** A — Landscape.

> ⚠️ `item_code` narrows lines but grouping stays per customer.

7. Sales Analysis — Customer Details

- **What:** Per customer → product (UOM): qty / value / VAT / total-incl-VAT + rollups.
- **Source:** `SII ⋈ SI ⋈ Item ⋈ Customer` , `GROUP BY customer, item_code, uom, is_return` .
- **Filters:** `company` , `from*` / `to*` , `customer` , `item_code` , `include_return` .
- **PDF:** A — Portrait.

> ⚠️ Granularity customer × item × UOM → row count can grow large.

8. Sales Analysis — Product Details

- **What:** Per product → customer → Invoice/Return rows, with rollups. Used by the `/product_wise_invoice_details` portal page (without Agent rows).
- **Source:** `GROUP BY item_code, uom, customer, si.name, is_return`. `build_rows(filters, include_return, include_agent)` builds the grouped layout.
- **Layout (per customer): Invoice** rows → **Return** rows → summary block (Customer Sales directly above Customer Returns). Section header rows (`is_section` : the "Invoice"/"Return" labels) render **bold**. *(2026-06-28)*
- **Filters:** `company` , `from*` / `to*` , `customer` , `item_code` , `include_return` .
- **PDF:** A — Landscape.

> ⚠️ No agent data source → "No Agent" label (suppressed when `include_agent=False`). Unchecked `include_return` never reaches the server (Frappe drops falsy filters → treated OFF).

9. Advance Tax TDS Details

- **What:** Supplier TDS withholding statement by category (Devanagari headers); per-section + grand totals.
- **Source:** `PI ⋈ PII ⋈ Supplier` ; `turnover = SUM(amount WHERE apply_tds=1)` , `tds = SUM(custom_tds_amount)` , `GROUP BY supplier, category, tax_id` . Category string regex-parsed → rate / khata / title.
- **Filters:** `company*` , `from*` / `to*` .
- **PDF:** B — shared helper + Prepared/Checked/Verified signature footer (print only).

10. Loan Summary

- **What:** Company-wise loan closing balances (short/long-term borrowings); ratio row; `Show Details` expands sub-accounts.
- **Source:** `tabAccount` (nested-set, CoA groups) → `tabGL Entry` : `SUM(credit-debit) WHERE posting_date <= to_date` .
- **Filters:** `company` , `to_date*` , `show_details` .
- **PDF:** B — shared helper.

> ⚠️ `from_date` is collected but **never used** — balances are cumulative to To Date, not a range.

11. One Lakh Above Transactions

- **What:** Parties (customers + suppliers) with taxable/exempt totals ≥ NPR 100,000.
- **Source:** SI and PI queries concatenated; `taxable = SUM(ABS(amount)) WHERE custom_vat_amount!=0` ; `HAVING ≥ 100000` (+ re-checked in Python).
- **Filters:** `company*` , `from*` / `to*` .
- **PDF:** C — stock Frappe print.

> ⚠️ `trade_name_type` hardcoded `'E'` ; no totals row.

Customizations & Portal

> **Grouped by the doctype you're working on.** When you add new work later, append it as a `####` task **under that doctype's heading** — don't start a new top-level section for a doctype that already appears here. Each task: **Purpose** (why) → **How it works** (the flow) → **Files** (where) → ⚠️ gotcha. Cross-doctype features are written under each doctype they touch (kept short on the secondary doctype, with a pointer to the primary).

Customer

(master, customer-facing portals)

Duplicate Name / Tax ID warning

Purpose: On saving a Customer, warn (Yes/No, non-blocking) if another Customer in the **same company** (`custom_company`) already uses the same `customer_name` or `tax_id` . Different company → silent. **How it works:** Client-side `validate` runs two async `frappe.db.get_value` lookups (name first, then `tax_id`), both filtered by `custom_company` and excluding self. Each hit pauses the save (`frappe.validated = false`) and shows a `frappe.confirm` — name → *"Do you want to create it again?"*, tax_id → *"Do you want to continue?"*. **Yes** proceeds, **No** aborts. The existing record's ID is a clickable link. (Shared with Supplier — same script.) **Files:** `public/js/party_duplicate_check.js` · `hooks.py` (`doctype_js`). See daily doc `docs/customer supplier item duplicate checks and default accounts.md` .

Default Accounting Row

Purpose: Auto-add one **Default Accounts** (`accounts` , child *Party Account*) row with **Company** pre-filled from `custom_company` . **How it works:** Client `refresh` + `custom_company` handlers: empty table → add one row with `company = custom_company` ; else keep the default row's company **synced** when the company changes (never clobbers a row where an account was already chosen, except its company). (Shared with Supplier — same script; Item has its own, see Item.) **Files:** `public/js/party_default_account.js` · `hooks.py` (`doctype_js`).

> ⚠️ Client-side only — `bench restart` + hard-refresh. `company_filter.js` may flash a "Removed N row(s)…" toast on company change; the row is re-synced so the end state is correct.

Place Order — Portal (`/place_order`)

Purpose: A logged-in customer places a Sales Order for **LP Gas** online. **How it works:** Guests → `/login` ; needs `Customer` role. The customer is resolved from the `Portal User` table (scoped to their linked customers). Fixed LP Gas item, UOM limited to cylinder sizes. Rates via `get_item_price` (`Item Price`); client computes amount + VAT (13% / 0% / manual) + totals. `create_sales_order` re-validates, builds and submits a **Sales Order** (`ignore_permissions` , `owner` = session user) → redirect `/orders` . **Files:** `templates/pages/place_order.py` (whitelisted `create_sales_order` , `get_customer_defaults` , `get_item_price` , `search_*`) · `place_order.html` (inline `PlaceOrder` JS).

Customer Statement — Portal (`/customer_statement`)

Purpose: A logged-in customer views their own account statement (Party-Ledger ledger) and downloads a Portrait PDF. **How it works:** `Customer` role required. `_get_portal_customers` + `_get_allowed_companies` drive the company/customer controls. **Every AJAX call goes through** `_resolve_request` — rejects companies/customers outside the user's set, and fills the party list with the user's own customers when none are chosen (empty party = "all parties" in Party Ledger → would leak data). `get_statement` reuses Party Ledger's `execute` ; one customer → flat layout (+ PAN/VAT), >1 → grouped. PDF reuses Party Ledger's `download_pdf` (Portrait, `capacity_override=76`). **Excluded accounts:** passes `exclude_account_patterns` to Party Ledger so deposit/security accounts never show or affect the balance — `Deposit Customers Cylinders (I)` (313101), `Record of Deposit Cylinders (1013)` (313102), `Security Deposit from Dealers` (313201); matched on `Account.account_name` LIKE (company-agnostic). **Files:** `templates/pages/customer_statement.py` (`_resolve_request` , `get_statement` , `download_pdf` , `EXCLUDE_ACCOUNT_PATTERNS`) · `customer_statement.html` · reuses `report/party_ledger/` .

> ⚠️ Dual AD + Nepali BS date pickers — AD is the source of truth; debounced reload (~350 ms).

Product Wise Invoice Details — Portal (`/product_wise_invoice_details`)

Purpose: A logged-in customer views the Product-wise Invoice Details for their own customer(s) — product → customer → Invoice/Return rows — **with returns, no Agent rows**. **How it works:** Same security as Customer Statement (reuses `_get_portal_customers` , `_get_allowed_companies` , `_resolve_request`). `get_data` builds filters (`company=[company]` , scoped `customer` , dates, `include_return`) and calls the report's `build_rows(filters, include_return, include_agent=False)` — `include_agent=False` drops the No Agent / Agent rows. Rows are flattened by `_shape` (product / customer / section / invoice / summary) for an HTML table. Filters: Company (defaults to Nepal Gas Karnali), AD (`YYYY-MM-DD`) + BS dates, Include Return toggle (default on). **Files:** `templates/pages/product_wise_invoice_details.py` (`get_data` , `_shape`) · `product_wise_invoice_details.html` · reuses `report/sales_analysis_product_wise_invoice_details/` .

> ⚠️ Default company prefers Nepal Gas Karnali; if a customer's company has no sales you'll see "No invoices found" — switch company.

Supplier

(master, supplier-facing portal)

Duplicate Name / Tax ID warning

Purpose: Same as Customer, for Supplier — same-company (`custom_company`) duplicate `supplier_name` or `tax_id` → non-blocking Yes/No warning. **How it works:** Identical mechanism and script as the Customer version (name → *"Do you want to create it again?"*, tax_id → *"Do you want to continue?"*). See **Customer** → **Duplicate Name** / **Tax ID warning** for the full flow. **Files:** `public/js/party_duplicate_check.js` · `hooks.py` (`doctype_js`).

Default Accounting Row

Purpose: Same as Customer — auto-add one **Default Accounts** (`accounts` , *Party Account*) row, Company from `custom_company` , synced on company change. **How it works:** Shared script (`party_default_account.js`). See **Customer** → **Default Accounting Row**. **Files:** `public/js/party_default_account.js` · `hooks.py` (`doctype_js`).

Request for Quotation — Supplier Portal override (`/rfq/<name>`)

Purpose: A supplier submits a Supplier Quotation with VAT / discount / attachments that ERPNext's default doesn't capture. **How it works:** `get_context` delegates to ERPNext (local `rfq.html`). `hooks.py` maps ERPNext's `create_supplier_quotation` → the local override, which additionally copies document-level discounts, sets per-line VAT custom fields (guarded by `item_meta.has_field`), runs `calculate_taxes_and_totals` , and re-links uploaded Files to the new quotation. **Files:** `templates/pages/rfq.py` (`create_supplier_quotation` , `_add_items` , `upload_portal_item_attachment`) · `rfq.html` · `hooks.py` (`override_whitelisted_methods`).

Item

(item master & price)

Default Accounting Row

Purpose: Auto-add one **Item Defaults** (`item_defaults` , child *Item Default*) row with **Company** pre-filled from the Item's `custom_company` . **How it works:** Same logic as the party version (`ensure_default_item_row`): empty table → add one row with `company = custom_company` ; else keep the default row's company synced on company change. **Files:** `public/js/item_default_account.js` · `hooks.py` (`doctype_js` Item). See daily doc `docs/customer supplier item duplicate checks and default accounts.md` .

Branch-wise Warehouse Auto-fetch

Purpose: Auto-set the line Warehouse on buying/selling docs to the correct branch warehouse — one item can have different warehouses per branch (and buying vs selling). **How it works:** Mapping is on the **Item** child table `custom_branch_wise_warehouse` (`custom_branch` , `custom_buying_warehouse` , `custom_selling_warehouse`); Item-level fields are the fallback. Resolution per item-add: branch row (matching the doc's `custom_branch`) → Item-level fallback → leave ERPNext's value. **Selling** (SI + Quotation/SO/DN): on `item_code` , temporarily wraps `frappe.call` so ERPNext's `get_item_details` response warehouse is replaced before it's written (avoids a race), plus a `before_save` sweep. **Buying** (Material Request): overrides `frm.events.get_item_data` . **Files:** `doctype/branch_wise_warehouse/` · `public/js/sales_invoice.js` , `sales_warehouse_common.js` , `material_request.js` · `hooks.py` .

> ⚠️ Only sets the warehouse when a mapping exists — a manually chosen warehouse is preserved.

Item Price Company Auto-assignment

Purpose: Stamp `company` on Item Price records auto-created by ERPNext, so price lists stay company-scoped. **How it works:** Monkey-patches `erpnext.stock.get_item_details.insert_item_price` , preserving standard behaviour but writing `company` (and/or `custom_company`) from the transaction `args` . One-time guard; wired via `before_request` . **Files:** `custom_code/Override/auto_insert_item_price.py` · `hooks.py` (`before_request`).

Sales Invoice & Selling

(SI / Quotation / SO / DN)

Credit Control

Purpose: Block new invoices for customers over their credit limits (overdue days, outstanding amount, unpaid-bill count). **How it works:** Reads Customer limits. On customer change → days + bill-count check (outstanding from `Customer Ledger Summary`). On `before_save` → adds the amount check (existing outstanding + this invoice's total vs limit). The client shows a warning dialog and rejects the save promise when blocked. **Files:** `custom_code/SalesInvoice/salesinvoice_customer.py` (whitelisted `check_customer_credit_limit_on_load` / `_on_save`) · `public/js/si.js`.

> ⚠️ Enforced via the client `before_save` path — the SI `validate` doc_event points at the taxes handler, not the credit `validate`. An unused `credit_control.py` variant exists.

Automatic Due Date

Purpose: `due_date = posting_date + Customer.custom_days_limit`. **How it works:** Client-side. `set_due_date_from_customer` runs on new-invoice refresh and on `customer` / `posting_date` change (wrapped in `setTimeout(…,0)` so it wins over ERPNext's handler). No `custom_days_limit` → 0 days. **Files:** `public/js/sales_invoice.js`.

Selling VAT & Tax Handling

Purpose: Consistent VAT + excise across SI / Quotation / SO / DN, default 13%. **How it works:** Per line `custom_vat_apply_on` ∈ { `VAT 13%`, `VAT 0%`, `Amount` }. `custom_total = base_net_amount + custom_excise_value`. VAT: 13% computed, 0% zeroed, Amount manual. `update_taxes_table` finds Excise (acct prefix `348204`) + VAT (prefix `VAT`), writes `Actual` rows, then `calculate_taxes_and_totals`. Returns: qty + `custom_vat_amount` forced negative. **Files:** `custom_code/common/selling_taxes_handler.py`, `custom_code/SalesInvoice/salesinvoice_taxes.py` · `public/js/selling_taxes_common.js`, `sales_invoice.js` · `hooks.py` doc_events.

Purchase & Buying

(PI / PO / PR / Supplier Quotation / MR / RFQ)

Purchase VAT & TDS Handling

Purpose: VAT + Excise + TDS on PI / PO / PR / Supplier Quotation; warehouse defaulting on MR / RFQ. **How it works:** VAT as on the selling side. **TDS** via `custom_tds_apply_on` (Percentage / Amount), only when `custom_tax_withholding_category_custom` is set AND the row's `apply_tds` is checked (rate from the Tax Withholding Category). `update_taxes_table` adds Excise/VAT (`Add`) + TDS (`Deduct`). Cross-document field mapping (SQ→PO→PR/PI) via `purchase_taxes_mapper.py`. Uses a **separate** `custom_tax_withholding_category_custom` field so ERPNext's native TDS engine stays out. **Files:** `custom_code/common/purchase_taxes_handler.py`, `purchase_taxes_mapper.py` · `public/js/purchase_taxes_common.js`, `pi.js` · `hooks.py` doc_events.

Cross-cutting

(applies to many / all doctypes)

Company-wise Filtering

Purpose: Restrict Link-field dropdowns (incl. child-table & Dynamic Link) to the document's company; block cross-company saves. **How it works:** `Company Filter Config` (`doctype_name`, `company_field`) + child `Company Filter Field` define what to filter. `get_filter_config` (Redis-cached) serves it. `company_filter.js` applies `frm.set_query()` per field (Dynamic Link → server query `search_link_by_company`); on company change, mismatched values are cleared. Server-side, `validate_company_matching` throws "Company Mismatch". Cache cleared on Config/Field save. **Files:** `custom_code/globalfilter/globalfilter.py` · `public/js/company_filter.js`, `global_filter.js` · doctypes `Company Filter Config` / `Field` · `hooks.py`.

Automatic Document Numbering (Voucher Number Settings)

Purpose: Human-facing `custom_document_no` + composite `custom_name` (e.g. `SGU-RC-000006-82/83`), auto or manual per transaction type. **How it works:** `AUTO_NUMBER_CONFIG` (discriminator field + qualifying types) decides auto vs manual. Next number = `max(matching custom_document_no) + 1`, matched by a `custom_name` LIKE pattern. Zero-pad width from **Voucher Number Settings Item** (`doctype_name → voucher_no_digits`, default 6). Client (`auto_update_document_no.js`) previews via whitelisted

`get_next_custom_document_no`; server re-runs authoritatively via `audit_file_manager` dispatchers. **Files:** `custom_code/Override/naming_series.py` · `public/js/auto_update_document_no.js` · `utils/audit_file_manager.py` · doctype `Voucher Number Settings` (+ child `Item`). **Backfill (console) — rebuild `custom_name` on existing records after the rules change:**

```
from avinashgroup_app.custom_code.voucher_no_console import (
    update_purchase_invoice_custom_names, update_journal_entry_custom_names,
    list_payment_entries_empty_custom_name,
)
update_purchase_invoice_custom_names()   # Purchase Invoice
update_journal_entry_custom_names()      # Journal Entry
list_payment_entries_empty_custom_name() # report PEs missing custom_name
```

Workflow Reject Reason

Purpose: Force a written reason on the workflow **Reject** action (audit trail). **How it works:** `before_workflow_action` opens a mandatory "Rejection Reason" dialog only for Reject; on submit calls whitelisted `set_reject_reason` → writes `custom_reason` (if the field exists, e.g. Purchase Order) else a document comment. `approval_workflow_auto.js` auto-binds it to any Dynamic-Approval doctype. **Files:** `custom_code/workflow.py`, `workflow_material_request.py` · `public/js/approval_workflow_common.js`, `approval_workflow_auto.js` · `custom/purchase_order.json` (`custom_reason`). **Setup (console):** install/refresh the app's workflows — `from avinashgroup_app.custom_code import update_workflows; update_workflows.run()`.

Audit Fields — Company / Naming Series / Created-Modified

Purpose: Stamp a consistent audit block on records: an **Audit** section + `custom_created_by` / `custom_created_on` / `custom_modified_by`; **`custom_company`** when the doctype has no company field; and for **master** doctypes also `custom_naming_series`. Filled automatically on save via the `set_audit_fields` doc_events. **How it works:** `AuditFieldsManager` (target list `AuditBase.doctypes` in `utils/audit_file_manager.py`; masters in `AuditBase.master_doctypes`) creates/removes the fields. `AuditEventMapper.get_doc_events()` wires the per-save handlers in `hooks.py`. **Files:** `utils/audit_file_manager.py` · `hooks.py` (`doc_events`). **Setup (console) — apply / verify / remove for specific doctype(s):**

```
from avinashgroup_app.utils.audit_file_manager import AuditFieldsManager, update_custom_company_reqd
AuditFieldsManager(["Item Price"]).create_fields()   # apply (Company + Naming Series + audit fields)
AuditFieldsManager(["Item Price"]).verify_fields()   # check what exists
AuditFieldsManager(["Item Price"]).remove_fields()   # remove them again
update_custom_company_reqd()                          # make custom_company mandatory across the set
```

> ⚠️ `AuditFieldsManager()` **with no list touches ~80 doctypes** — always pass an explicit list (e.g. `["Item Price"]`). `remove_fields()` deletes `audit_tab`, `custom_created_by/on`, `custom_modified_by`, `custom_company`, `custom_naming_series` — use it to undo fields a doctype picked up by mistake. Per-site: run on each site; `bench restart` on live.
> **Created By/On — alt installer + backfill existing rows (console):**

```
from avinashgroup_app.custom_code import api
api.create_created_by_and_created_on_fields()   # add the fields
api.populate_created_by_and_created_on()        # backfill existing rows
```

Company-field Lock

Purpose: Make `company` / `custom_company` read-only once a record is saved, so it can't be changed after creation. **How it works:** Sets `read_only_depends_on = eval:!doc.__islocal` on the company field across a configured set of doctypes. **Files:** `custom_code/Override/setup_company_lock.py`, `company_field_lock.py`. **Setup (console):**

```
from avinashgroup_app.custom_code.Override import setup_company_lock
setup_company_lock.quick_setup()                  # apply to the configured set
setup_company_lock.verify_setup()                 # check status
setup_company_lock.fix_specific_doctype("Customer")
setup_company_lock.show_help()                    # prints usage
```

Regional Document-Deletion Guard

Purpose: Block deletion of regionally-significant submitted documents (audit/compliance). **How it works:** `check_deletion_permission` (doc_event) enforces it; `apply_patch()` installs the guard. **Files:** `custom_code/regional_deletion_override.py`. **Setup (console):** `from avinashgroup_app.custom_code import regional_deletion_override; regional_deletion_override.apply_patch()`.